1. Iterator

1.1 The Essence of an Iterator

An iterator is a **unified traversal interface**.

1.2 Key Methods for Using Iterators in Different Classes

Common container-side interfaces

- `begin()` : returns the starting iterator
- `end()` : returns the ending iterator (the position **one past the last element**)

Common iterator-side interfaces

- `operator++()` : advance to the next element
- `operator==()` : check whether two iterators are equal
- `operator!=()` : check whether two iterators are not equal
- `operator*()` : dereference and get the element at the current position
- `operator->()` : access a member of the current element

Standard traversal template

stlList.cpp

```cpp
#include <list>
#include <string>
#include <iostream>

struct Animal {
  std::string name, food;
  bool big;
  Animal(std::string name = "blob", std::string food = "you", bool big = true) :
    name(name), food(food), big(big) { /* none */ }
};

int main() {
  Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
  std::list<Animal> zoo;

  zoo.push_back(g);
  zoo.push_back(p);   // std::list's insertAtEnd
  zoo.push_back(b);

  for ( std::list<Animal>::iterator it = zoo.begin(); it != zoo.end(); it++ ) {
    std::cout << (*it).name << " " << (*it).food << std::endl;
  }

  return 0;
}
```

1.3 `(*it).x` vs `it->x`

If an iterator points to an object, these two forms are usually equivalent:

- `(*it).x`
- `it->x`

Be careful with operator precedence:

- `.` has higher precedence than `*`
- So `*it.name` is wrong

You must write:

- `(*it).name`

1.4 Implementing a Customized Iterator

Queue.h

```
 4  template <class QE>
 5  class Queue {
 6    public:
 7      class QueueIterator : public std::iterator<std::bidirectional_iterator_tag, T> {
 8        public:
 9          QueueIterator(unsigned index);
10          QueueIterator& operator++();
11          bool operator==(const QueueIterator &other);
12          bool operator!=(const QueueIterator &other);
13          QE& operator*();
14          QE* operator->();
15        private:
16          unsigned location_;
17      }
18
19                    Access to location in the data structure.
20
21
22    private:
23      QE* arr_; unsigned capacity_, count_, entry_, exit_;
24
25            Array based.
26  };
```

stlList.cpp

```cpp
#include <Queue.h>
#include <string>
#include <iostream>

struct Animal {
  std::string name, food;
  bool big;
  Animal(std::string name = "blob", std::string food = "you", bool big = true) :
    name(name), food(food), big(big) { /* none */ }
}

int main() {
  Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
  Queue<Animal> zoo;

  zoo.push_back(g);
  zoo.push_back(p);
  zoo.push_back(b);

  for ( Queue<Animal>::QueueIterator it = zoo.begin(); it != zoo.end(); it++ ) {
    std::cout << (*it).name << " " << (*it).food << std::endl;
  }

  return 0;
}
```

2. Recursion

2.1 Basic Idea of Recursion

Recursion means:

- A function directly or indirectly calls itself.

For recursion to work, it must contain two parts:

Base case

The smallest problem, answered directly without further self-calls.

Recursive case

Reduce the current problem into a smaller problem of the same type, then call itself to solve it.

2.2 Two Core Rules of Recursion

Rule 1: There must be a base case

Rule 2: Every recursive call must get closer to the base case

2.3 Implementing Simple Recursive Functions

Example 1: Factorial

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

3. List Array

3.1 Run Time

Operation	Unsorted Array List	Sorted Array List
`insertAtFront`	$O(1)$	$O(1)$
`insertAtIndex(i)`	$O(n)$	$O(n)$
`removeAtIndex(i)`	$O(n)$	$O(n)$
`insertAfterElement(x)`	$O(n)$	$O(n)$
`removeAfterElement(x)`	$O(n)$	$O(n)$
`findIndex(i)`	$O(1)$	$O(1)$
`findData(x)`	$O(n)$	$O(\log n)$

`insertAtFront`：
We need to consider the resize case, for both unsorted and sorted array, O(1).

`insertAtIndex(i)`：

- unsorted: find: O(1), move: O(n)
- sorted: find: O(1), move: O(n)

`removeAtIndex(i)`：

- unsorted: find: O(1), move: O(n)
- sorted: find: O(1), move: O(n)

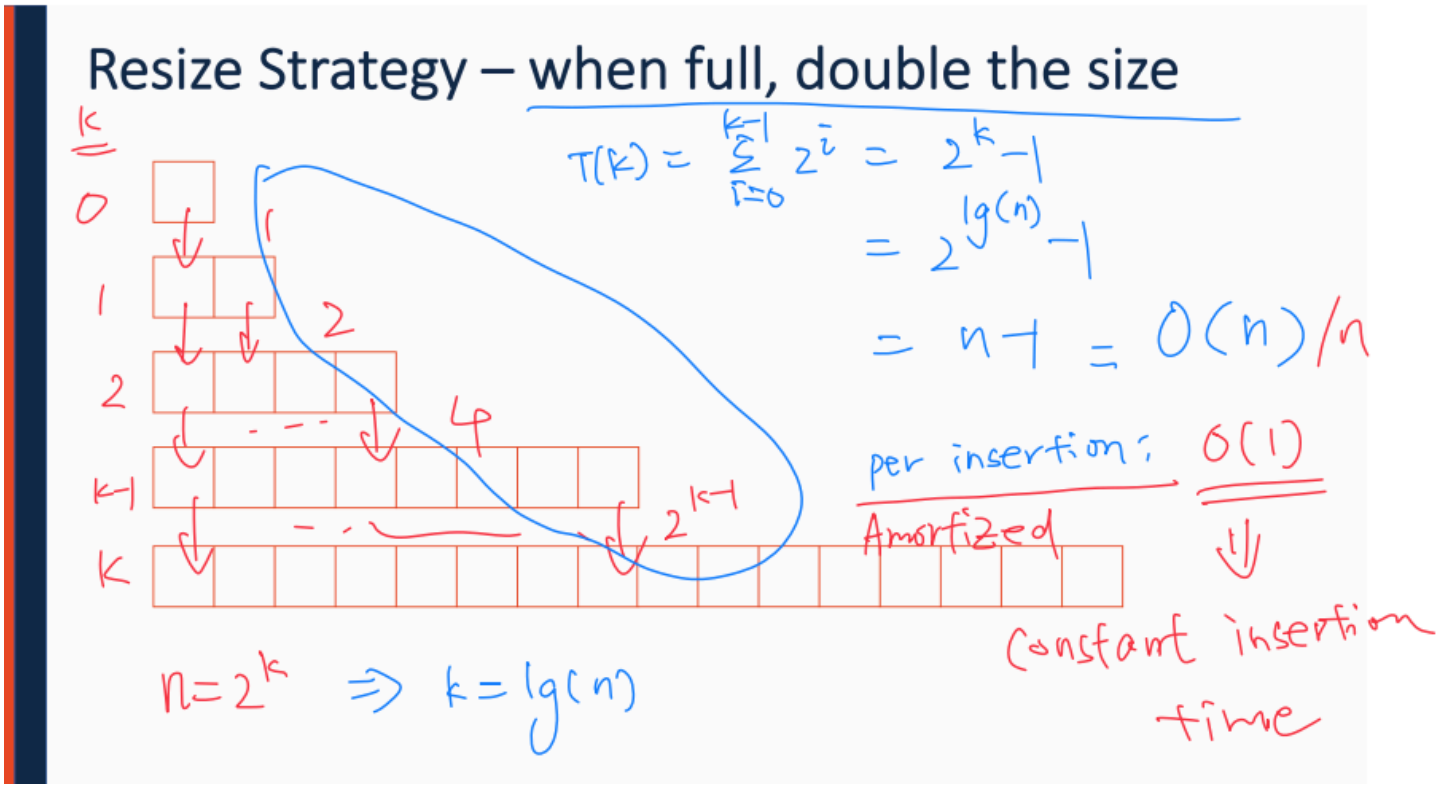
`insertAfterElement(x)`：

- unsorted: find: O(n), move: O(n)
- sorted: find: <O(n), move: O(n)

`removeAfterElement(x)`：

- unsorted: find: O(n), move: O(n)
- sorted: find: <O(n), move: O(n)

3.2 Resize Strategy



4. Linked List

4.1 Run Time

Operation	Unsorted Linked List	Sorted Linked List
`insertAtFront`	$O(1)$	$O(1)$
`insertAtIndex(i)`	$O(n)$	$O(n)$
`removeAtIndex(i)`	$O(n)$	$O(n)$
`insertAfterElement(x)`	$O(n)$	$O(n)$
`removeAfterElement(x)`	$O(n)$	$O(n)$
`findIndex(i)`	$O(n)$	$O(n)$
`findData(x)`	$O(n)$	worst $O(n)$

`insertAtFront` :

```
newNode->next = head;
head = newNode;
```

`insertAtIndex(i)` :

- Finding the position: `O(n)`
- Linking itself: `O(1)`
- Total: `O(n)`

`removeAtIndex(i)` :

- Finding the position: `O(n)`
- Remove: `O(1)`
- Total: `O(n)`

```
prev->next = target->next;
```

`insertAfterElement(x)` :

- Search: `O(n)`

- Link insertion: `O(1)`
- Total: `O(n)`

`removeAfterElement(x)` :

- Find target element: `O(n)`
- Delete successor: `O(1)`
- Total: `O(n)`